HealthDevice: Engineering Architecture Manual

Complete Technical Guide to Software Compilation, Critical Source Code Analysis & AI Posture Cam Engine

Wearable Firmware | Flutter Multiplatform | MediaPipe AI Pose Estimation | Inno Setup Packaging | ESP32-C3 Super Mini

---

Section 1: How the Software & Firmware Are Built

The HealthDevice platform comprises three coordinated software tiers: the embedded bare-metal ESP32-C3 firmware, a cross-platform Flutter application (targeting Android and Windows 64-bit), and an autonomous Python computer-vision posture tracker.

1.1 Wearable Firmware Build Pipeline (ESP32-C3)

The firmware is written in C++ using the Arduino-ESP32 framework for Espressif's RISC-V architecture. Compilation is orchestrated using the Arduino IDE or `arduino-cli` against the `ESP32C3 Dev Module` target definition.

> **Firmware Build Pre-requisites & Compiler Settings**
>
> - **Target Core:** `esp32` by Espressif Systems (v2.0.x or v3.0.x).
> - **Board Selection:** `ESP32C3 Dev Module`.
> - **CPU Frequency:** 160 MHz (Standard) or 80 MHz (Low-Power Profile).
> - **Flash Frequency & Mode:** 80 MHz, DIO (Dual I/O).
> - **USB CDC on Boot:** Enabled (for serial diagnostics over native USB).
> - **Required External Libraries:** Adafruit GFX, Adafruit SSD1306, SparkFun MAX3010x Pulse and Proximity Sensor Library.

Compilation Step: Run `Verify / Upload` in the Arduino IDE. The compiler links `HealthDevice.ino` with 14 modular C++ implementation units, assembling an optimized binary image into flash memory partitions (App, NVS, SPIFFS).

1.2 Mobile Android Build Pipeline (Release APK)

The Android application is built from the root Flutter workspace (`healthdeviceapp`). It targets Android API Level 21 through 34+ using Gradle 8.x and Android Gradle Plugin (AGP).

```
# Command executed in healthdeviceapp directory:
flutter build apk --release

# Output path:
build/app/outputs/flutter-apk/app-release.apk (56.6 MB)
Copied to: D:\SG\MAIN FILES\Health Device.apk
```

Key Build Mechanics:

- **Tree Shaking:** Unused vector glyphs from `MaterialIcons-Regular.otf` are tree-shaken (reduced from 1.6 MB to 22 KB, a 98.7% payload reduction).
- **R8 Minification & Bytecode Optimization:** Unused Java/Kotlin classes from BLE and network plugins are stripped.
- **AOT (Ahead-Of-Time) Native Compilation:** Dart code is compiled into native ARM64 and ARMv7 machine code (`libapp.so`), ensuring zero JIT warm-up latency.

1.3 Windows Desktop Build Pipeline (64-bit Standalone)

The Windows desktop software compiles via Microsoft Visual Studio 2022 C++ toolchain (MSVC) and CMake into a native 64-bit Win32 executable.

```
# Command executed in healthdeviceapp directory:
flutter build windows --release

# Output path:
build/windows/x64/runner/Release/Health Device.exe
```

Desktop Artifact Bundling: Unlike Android, a Windows Flutter app requires its runtime engine and plugin DLLs alongside the main executable:

- `Health Device.exe` — Native C++ Win32 shell hosting the Flutter embedder window.
- `flutter_windows.dll` — Core Skia rendering engine and platform dispatcher.
- `flutter_blue_plus_winrt_plugin.dll` — Windows Runtime (WinRT) Bluetooth Low Energy bridge.
- `app_links_plugin.dll` & `url_launcher_windows_plugin.dll` — Deep link and external browser dispatchers.
- `data/` — Contains compiled Dart AOT binary (`app.so`), fonts, shaders, and assets.

1.4 Inno Setup Installer Packaging (Windows)

To distribute the PC software as an industry-standard Windows installer, an Inno Setup script (`windows_installer_script.iss`) is compiled using `D:\Inno Setup 7\ISCC.exe` .

```
# Command to compile installer:
& "D:\Inno Setup 7\ISCC.exe" "D:\SG\MAIN FILES\healthdeviceapp\windows_installer_script.iss"

# Output Executable:
D:\HealthDevice_Setup_x64.exe (16.17 MB)
```

Packaging Features:

- **LZMA2/Ultra64 Solid Compression:** Compresses the 80 MB bundle down to 16.17 MB.
- **Directory Staging:** Bundles the Flutter release runtime, the Python AI posture tracker, voice prompts, and MediaPipe AI models into `{autopf}\Health Device`.
- **Automated Shortcuts & Uninstaller:** Generates Desktop and Start Menu icons with `app_icon.ico`, and registers clean uninstallation entries in Windows Control Panel.

Section 2: Critical Code Analysis & Engineering Walkthrough

This section dissects the vital code mechanisms in both the firmware and client software that ensure real-time responsiveness, crash-proof arbitration, and persistent multi-alarm illumination.

2.1 Firmware: Non-Blocking Delta-Time Execution Engine

Wearable firmware must never freeze or execute blocking delays. In `HealthDevice.ino`, all operational subsystems execute on cooperative delta-time schedules:

```
// HealthDevice.ino: Main Execution Loop
void loop() {
  bleUpdate();               // Maintain incoming BLE characteristic buffers
  ButtonEvent ev = button::update(); // Read asynchronous debounce FSM
  if (ev != BTN_NONE) {
    handleButtonEvent(ev);   // Immediate click routing
    if (ev != BTN_LOW_POWER_HOLD) display::noteActivity();
  }
  display::updateTimeout();  // Handle auto-sleep OLED dimming
  sensors::update();         // Read MAX30102 PPG FIFO and MPU-6500 IMU
  timeManager::update();     // Maintain virtual RTC and NTP synchronization
  alarms::update();          // Check 24-hour match across all 5 alarms
  reminders::update();       // Hydration and movement schedule triggers
  wellness::update();        // Activity and wellness tracker
  updatePostureAlert();      // Posture degradation escalation (30s / 60s)
  alerts::update();          // Non-blocking buzzer pattern step generator
  updateAlertArbitration();  // Priority stack screen takeover arbitration
  updateStatusLed();         // Modulate RGB LED PWM hardware channels
  display::render();         // Push dirty pixel buffer to SSD1306 OLED
  delay(1);                  // Yield CPU to FreeRTOS IDLE task (WDT feed)
}
```

> **Why this matters:**
> The `delay(1)` at the end yields control to the underlying ESP-IDF FreeRTOS scheduler, feeding the hardware Watchdog Timer (WDT) and allowing the Wi-Fi/BLE radio stack to handle background interrupts without dropping packets.

2.2 Firmware: PPG Pulse & Blood Oxygen (SpO2) Calculation

In `sensors.cpp`, the optical sensor measures minute volumetric blood changes in capillary tissue:

```
// sensors.cpp: Optical PPG Peak Detection & Ratio of Ratios
// 1. Finger presence verification
if (irValue < HR_FINGER_THRESHOLD) { // 100,000 threshold
  fingerDetected = false;
  return;
}

// 2. Optical Ratio of Ratios calculation (R)
float red_ac = red_max - red_min;
float red_dc = (red_max + red_min) / 2.0f;
float ir_ac  = ir_max - ir_min;
float ir_dc  = (ir_max + ir_min) / 2.0f;

if (ir_ac > 0 && ir_dc > 0 && red_dc > 0) {
  float R = (red_ac / red_dc) / (ir_ac / ir_dc);

  // Empirical pulse-oximetry calibration polynomial
  float calculatedSpo2 = 110.0f - 25.0f * R;
  spo2 = constrain((int)calculatedSpo2, 80, 100);
}

// 3. Heart Rate peak-to-peak timing
if (checkForBeat(irValue)) {
  unsigned long now = millis();
  unsigned long delta = now - lastBeatMs;
  lastBeatMs = now;

  if (delta > 300 && delta < 2000) { // 30 - 200 BPM valid window
    int rawBpm = 60000 / delta;
    bpm = (bpm * 3 + rawBpm) / 4;    // 4-sample moving average filter
  }
}
```

2.3 Firmware: 4-Tier Alert Arbitration & Screen Takeover

To ensure an incoming wake-up alarm or timer alert is never suppressed by a low-priority notification, `updateAlertArbitration()` in `HealthDevice.ino` enforces strict hierarchical priority:

```
// HealthDevice.ino: Alert Priority Arbitration Stack
static void updateAlertArbitration() {
  DeviceScreen current = display::getCurrentScreen();
  AlertPriority active = alerts::getActive();

  // Priority 1: ALARM has absolute top priority
  if (active == ALERT_ALARM) {
    if (current != SCR_ALERT_ALARM) {
      screenBeforeAlert = (current == SCR_ALERT_POSTURE ||
                           current == SCR_ALERT_REMINDER ||
                           current == SCR_ALERT_TIMER_DONE) ? SCR_HOME : current;
      display::setScreen(SCR_ALERT_ALARM);
      display::wake(); // Force display awake even if timed out
    }
    return;
  }

  // Priority 2 - 4: Handled only when no higher overlay is blocking
  bool onOverlay = (current == SCR_ALERT_ALARM || current == SCR_ALERT_REMINDER ||
                    current == SCR_ALERT_TIMER_DONE);
  if (!onOverlay) {
    if (active == ALERT_TIMER) {
      display::setScreen(SCR_ALERT_TIMER_DONE);
      display::wake();
      if (settings::getSoundEnabled()) alerts::beepPattern(4, 200, 200);
    } else if (active == ALERT_REMINDER) {
      display::setScreen(SCR_ALERT_REMINDER);
      display::wake();
      if (settings::getSoundEnabled()) alerts::beepPattern(2, 150, 150);
    }
  }
}
```

2.4 Firmware: Multi-Alarm Engine with Custom 24-Bit RGB Illumination

The alarm engine in `alarms.cpp` and `HealthDevice.ino` stores 5 independent alarms, custom RGB color channels, and recurrence masks in NVS flash:

```
// alarms.cpp: Day Matching & Zero-Day Fallback Logic
bool dayMatches = list[i].days[dt.weekday];
if (!anyDaySelected(list[i].days)) {
  dayMatches = true; // Unconstrained alarms fire every day
}

if (minuteOfDay == target && lastFiredMinuteOfDay[i] != minuteOfDay) {
  lastFiredMinuteOfDay[i] = minuteOfDay; // Prevent re-triggering in same minute
  ringingIndex = i;
  alerts::raise(ALERT_ALARM);
  alerts::buzzerContinuousStart();
  break;
}

// HealthDevice.ino: Per-Alarm Dynamic RGB LED Illumination
if (current == SCR_ALERT_ALARM || alarms::isRinging()) {
  if (now - lastAlarmLedBlinkMs >= 200) { // Rapid 200ms alarm beacon
    lastAlarmLedBlinkMs = now;
    alarmLedBlinkState = !alarmLedBlinkState;
  }
  if (alarmLedBlinkState) {
    const Alarm& a = alarms::getRinging();
    if (a.r == 0 && a.g == 0 && a.b == 0) {
      setStatusLedRgb(255, 0, 0); // Safe fallback to bright red
    } else {
      setStatusLedRgb(a.r, a.g, a.b); // Drive custom chosen alarm color
    }
  } else {
    setStatusLed(false, false, false);
  }
  return;
}
```

2.5 Software: BLE Command Serialization Pipeline (Flutter)

In `lib/services/ble_command_service.dart`, alarm configurations created in the Flutter UI (including the interactive RGB color wheel) are serialized into the hardware command format:

```dart
// ble_command_service.dart: Serializing Alarms with RGB Color Channels
Future<BleCommandResult> syncAlarms(List<AlarmItem> alarms) async {
  if (!_bleManager.isConnected) return const BleCommandResult.disconnected();

  try {
    // 1. Flush existing hardware alarms
    await sendRawCommand('CLEAR_ALARMS');
    await Future.delayed(const Duration(milliseconds: 50));

    // 2. Transmit each alarm with day mask and RGB color
    for (final alarm in alarms) {
      final parts = _parseHourMinute(alarm.time);
      final h = parts[0];
      final m = parts[1];
      final en = alarm.enabled ? 1 : 0;
      final daysMask = _buildDaysMask(alarm.repeatDays);
      final r = alarm.ledColor.red;
      final g = alarm.ledColor.green;
      final b = alarm.ledColor.blue;

      // Protocol format: ADD_ALARM:H,M,EN,DAYS,R,G,B
      await sendRawCommand('ADD_ALARM:$h,$m,$en,$daysMask,$r,$g,$b');
      await Future.delayed(const Duration(milliseconds: 50));
    }
    return const BleCommandResult.synced(message: 'Alarms successfully synchronized.');
  } catch (e) {
    return BleCommandResult.syncFailed(message: 'Sync error: $e');
  }
}
```

Section 3: Deep Dive into the AI Posture Camera Engine

While the wearable capsule tracks torso tilt using its onboard 3-axis accelerometer, the desktop companion includes a dedicated computer vision AI posture tracker (`ai_posture_tracker/posture_tracker.py`). This engine uses standard webcam video to perform optical ergonomic analysis of cervical and spinal alignment.

```
+-------------------------------------------------------------------------------+ | AI POSTURE CAMERA COMPUTER VISION PIPELINE | +-------------------------------------------------------------------------------+ | | | [ Web Camera ] --> (OpenCV VideoCapture) --> BGR to RGB Frame Conversion | | | | [ MediaPipe Pose Landmarker Lite ] --> 33 3D Keypoints Extracted in Real-Time | | | | +--> Landmark 0: Nose Tip | | +--> Landmarks 7 & 8: Left / Right Ear Tragus | | +--> Landmarks 11 & 12: Left / Right Acromion (Shoulders) | | | | [ Ergonomic Geometry Engine ] | | | | +--> 1. Vertical Spine Compression Ratio (Slouch / Spine Sinking) | | +--> 2. Craniovertebral Angle & Ear-to-Shoulder Distance (Forward Head) | | +--> 3. Horizontal Acromion Tilt Angle (Shoulder Imbalance) | | | | [ Posture Evaluation State Machine ] | | | | | +--> Absence Filter: If landmarks absent, freeze timer (No False Alerts) | | +--> Grace Period: 4 seconds of bad posture required to trigger alert | | +--> Alert Trigger: Play gender-matched studio voice prompt | | +--> Keyboard Snooze: Global Win32 Ctrl+B listener mutes for 30 seconds | | | | [ IPC & Telemetry HTTP Server ] | | +--> Serves GET /status and POST /stop to Flutter application on port | +-------------------------------------------------------------------------------+
```

3.1 Key Skeletal Landmarks Tracked

The engine loads Google's `pose_landmarker_lite.task` model ($5.7 ext{ MB}$ quantized neural network). It extracts 33 body landmarks with 3D normalized coordinates (x, y, z) and visibility scores. The posture engine isolates 5 critical points:

Landmark ID	Anatomical Name	Coordinate Representation	Ergonomic Purpose
0	Nose Tip	(x_0, y_0, z_0)	Head forward trajectory and downward chin drop.
7	Left Ear (Tragus)	(x_7, y_7, z_7)	Cervical spine curvature and ear-shoulder vertical alignment.
8	Right Ear (Tragus)	(x_8, y_8, z_8)	Cervical spine curvature (used symmetrically with left ear).
11	Left Shoulder (Acromion)	(x_{11}, y_{11}, z_{11})	Torso vertical height baseline and shoulder horizontal axis.
12	Right Shoulder (Acromion)	(x_{12}, y_{12}, z_{12})	Torso vertical height baseline and shoulder horizontal axis.

3.2 Ergonomic Mathematical Formulas

1. Shoulder Midpoint & Torso Baseline

The center of the shoulder girdle acts as the thoracic origin point:

```
# Python: Calculating Shoulder Midpoint
shoulder_mid_x = (l_sh.x + r_sh.x) / 2.0
shoulder_mid_y = (l_sh.y + r_sh.y) / 2.0
shoulder_width = math.dist((l_sh.x, l_sh.y), (r_sh.x, r_sh.y))
```

2. Forward Head Posture (FHP) & Neck Compression Ratio

Forward head posture occurs when the ears translate forward and downward relative to the shoulders. The normalized vertical neck height $H_{ ext{neck}}$ is calculated relative to shoulder width $W_{ ext{shoulder}}$ to eliminate camera-distance dependency:

```
# Python: Distance-Independent Neck Ratio
ear_mid_y = (l_ear.y + r_ear.y) / 2.0
current_neck_dist = (shoulder_mid_y - ear_mid_y) / shoulder_width

# Comparison against calibrated baseline:
neck_drop_ratio = current_neck_dist / baseline_neck_dist
if neck_drop_ratio < 0.82: # More than 18% vertical head compression
    is_forward_head = True
```

3. Shoulder Horizontal Tilt Angle (ϕ)

Uneven shoulders indicate scoliosis-like leaning or resting on an armrest:

```
# Python: Shoulder Tilt Calculation
delta_y = r_sh.y - l_sh.y
delta_x = r_sh.x - l_sh.x
tilt_angle_rad = math.atan2(delta_y, delta_x)
tilt_angle_deg = abs(math.degrees(tilt_angle_rad))

if tilt_angle_deg > 7.5: # More than 7.5 degrees lateral tilt
    is_uneven_shoulders = True
```

3.3 Gender-Adaptive Studio Voice Prompt Synthesis

Rather than generic beeps, the engine features natural studio-quality human voice prompts. To maintain appropriate personalized etiquette, `get_current_gender_voice_mode()` queries the user's local profile store (`settings_store.json`):

```
# posture_tracker.py: Biological Sex Profile Resolution
def get_current_gender_voice_mode():
    store_path = os.path.join(local_app_data, 'HealthDeviceApp', 'settings_store.json')
    if os.path.exists(store_path):
        with open(store_path, 'r', encoding='utf-8') as sf:
            store_data = json.load(sf)
            bio_sex = store_data.get('hd_health_profile', {}).get('biologicalSex', '').lower()
            if 'female' in bio_sex or 'woman' in bio_sex:
                return 'maam'    # Plays voice_maam.mp3: "Straighten your back, ma'am"
            elif 'male' in bio_sex or 'man' in bio_sex:
                return 'sir'     # Plays voice_sir.mp3: "Please sit straight, sir"
            else:
                return 'neutral' # Plays voice_neutral.mp3: "Posture check: sit straight"
    return 'neutral'
```

3.4 Global Keyboard Snooze & Win32 Interoperability

When slouching during typing or work, taking hands off the keyboard to click a button is disruptive. The tracker registers a system-wide global hotkey (**Ctrl + B**) using the Win32 API (`RegisterHotKey`):

```
# posture_tracker.py: System-Wide Global Hotkey Listener (Ctrl + B)
def hotkey_listener_thread(state):
    user32 = ctypes.windll.user32
    HOTKEY_ID = 1
    MOD_CONTROL = 0x0002
    VK_B = 0x42 # 'B' virtual key code

    if user32.RegisterHotKey(None, HOTKEY_ID, MOD_CONTROL, VK_B):
        msg = ctypes.wintypes.MSG()
        while state.running:
            if user32.GetMessageA(ctypes.byref(msg), None, 0, 0) != 0:
                if msg.message == 0x0312 and msg.wParam == HOTKEY_ID: // WM_HOTKEY
                    state.snooze_until = time.time() + 30.0 # 30s snooze
                    state.status = "Snoozed (Ctrl+B)"
                user32.TranslateMessage(ctypes.byref(msg))
                user32.DispatchMessageA(ctypes.byref(msg))
        user32.UnregisterHotKey(None, HOTKEY_ID)
```

3.5 Headless Background Operation & Inter-Process Communication (IPC)

The tracker supports full background execution without keeping an active console window open:

- **Launch Via `pythonw.exe` :** Eliminates command-line window flicker.
- **Safe Window Close:** Clicking the 'X' button on the OpenCV preview window toggles `show_window = False` but keeps the background tracker active.
- **Embedded HTTP Control Server:** The script runs a lightweight `BaseHTTPRequestHandler` on an ephemeral local port, exposing:
  - `GET /status` — Returns live JSON containing current posture score, slouch duration, and camera status.
  - `POST /recalibrate` — Re-captures baseline vector without restarting camera.

- `POST /stop` — Allows the Flutter desktop application to cleanly terminate the Python process on app close.

Section 4: Technical Summary & Architectural Comparison

Architectural Dimension	Wearable Hardware Capsule	Desktop AI Posture Vision
Core Sensor	MPU-6500 6-Axis IMU (Hardware Accelerometer)	Standard USB / Integrated Laptop Web Camera
Computational Mechanism	3D Gravity Vector Dot Product ($ heta = rccos(\hat{A} \cdot \hat{A}_0)$)	MediaPipe Neural Pose Landmark Detection (33 3D joints)
Primary Tracking Focus	Gross Torso / Thoracic spine forward and lateral tilt	Craniovertebral angle, forward head posture, shoulder height
User Mobility	Fully portable (walk, run, sit, indoors, outdoors)	Stationary desk / office workspace
Escalation Mechanism	30s Solid Red LED $ o$ 60s Double-Beep Piezo buzzer	4s grace period $ o$ Personalized human voice prompt
Snooze Method	Physical hardware push button on GPIO 3	Global keyboard hotkey (**Ctrl + B**)

Synergistic Operation

The two systems complement each other. When working at a desk, the AI Posture Camera detects subtle forward-head neck compression that an accelerometer on the chest or wrist might miss. When walking or away from the desk, the wearable hardware capsule maintains step counts, heart rate monitoring, continuous IMU activity tracking, and active alarm scheduling.

© 2026 HealthDevice Development Team — Shaurya Gupta • All Rights Reserved.